Goldman Sachs
Transformative Journey with Image Mode for RHEL

Modernizing Enterprise OS Management at Scale

Mike Hanulec
Vice President - Goldman Sachs

Colin Walters
Architect - Red Hat

Mohan Shash
Product Owner - Red Hat

Red Hat

Standardizing and innovating with containers

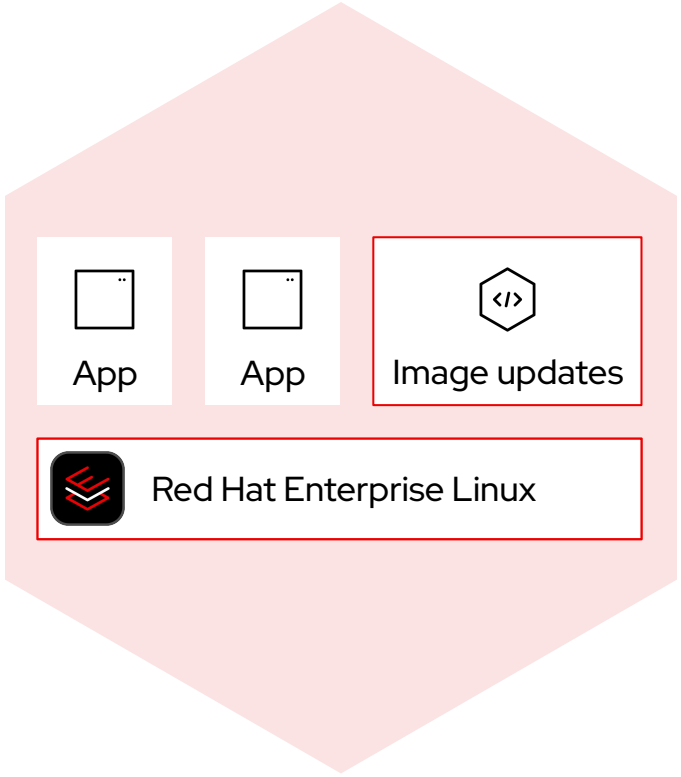


Image mode for Red Hat Enterprise Linux is a simple, consistent approach to build, deploy, and manage the operating system using container technologies.

Now you can manage the operating system with the same tools and workflows as applications, promoting a common experience and language across teams.

Contain drift and accelerate delivery

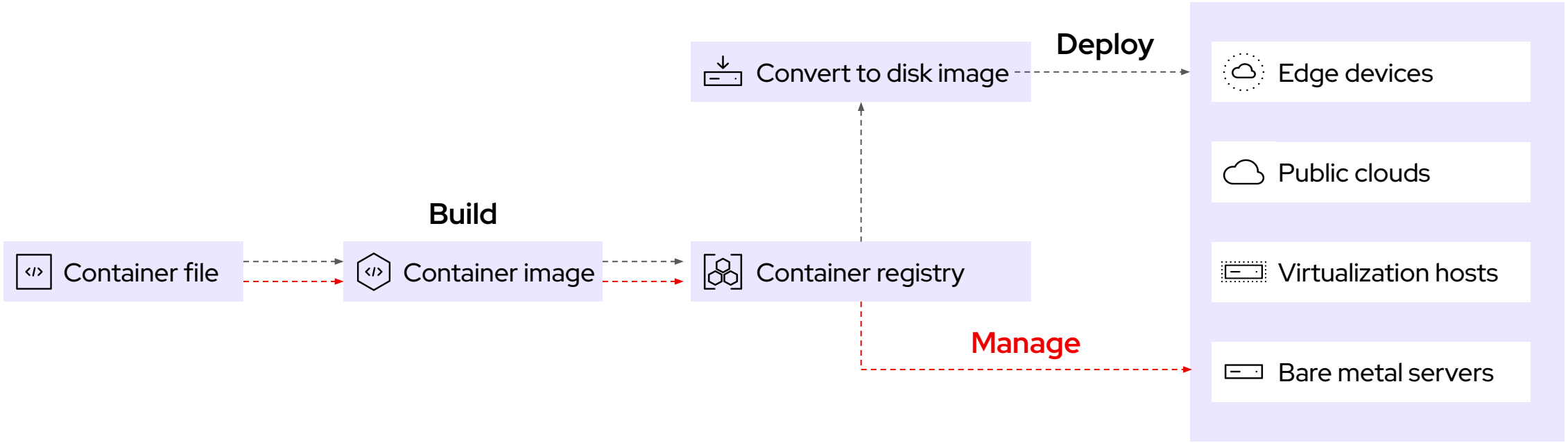
Container tools and technologies

Image mode for Red Hat Enterprise Linux, because systems should be as easy to update as smartphones

- **Speed time to market**
  DevOps and continuous integration and delivery (CI/CD) practices now include the OS
- **Streamline operations**
  Automate updates and rollbacks—just like your smartphone
- **Enhance security**
  Reduce your attack surface with immutable system images
- **Simplify appliance creation**
  Combine the OS with apps and drivers for faster development and delivery

Image mode for Red Hat Enterprise Linux

Simple. Consistent. Anywhere.



Goldman Sachs's Journey

Enterprise Scale

Managing thousands of mission-critical RHEL systems across global markets with absolute consistency.

Real Production

This isn't a POC; it's a fundamental architectural shift for years of accumulated infrastructure debt.

Legacy Complexity

Modernizing bespoke image-building workflows while maintaining strict financial sector compliance.

Measurable Wins

Proven 8-10x faster image creation and 30x faster installation times in real-world environments.

Dramatic Performance Improvements

Operation	Package Mode	Image Mode for RHEL	Improvement
Image Creation	Up to 1 hour	~7 minutes	**8-10x Faster**
System Installation	~45 minutes	~90 seconds	**30x Faster**
Updates / Upgrades	Highly Variable	~90 seconds	**Consistent**
Downgrades/Rollbacks	Complex/Manual	~90 seconds	**Seamless**

Quality of life improvement for internal users through faster patching cycles

Breaking Years of Tech Debt

Replaced Bespoke Tools

Replaced fragile, image building systems with standard Podman workflows.

Standard Infrastructure

Shifted from custom delivery methods to enterprise OCI registries.

Unified Skills

Common tools across OS and App container deployments reduce specialized training.

Community Support

Leveraging a Container ecosystem rather than maintaining isolated custom scripts.

Transformative Business Impact

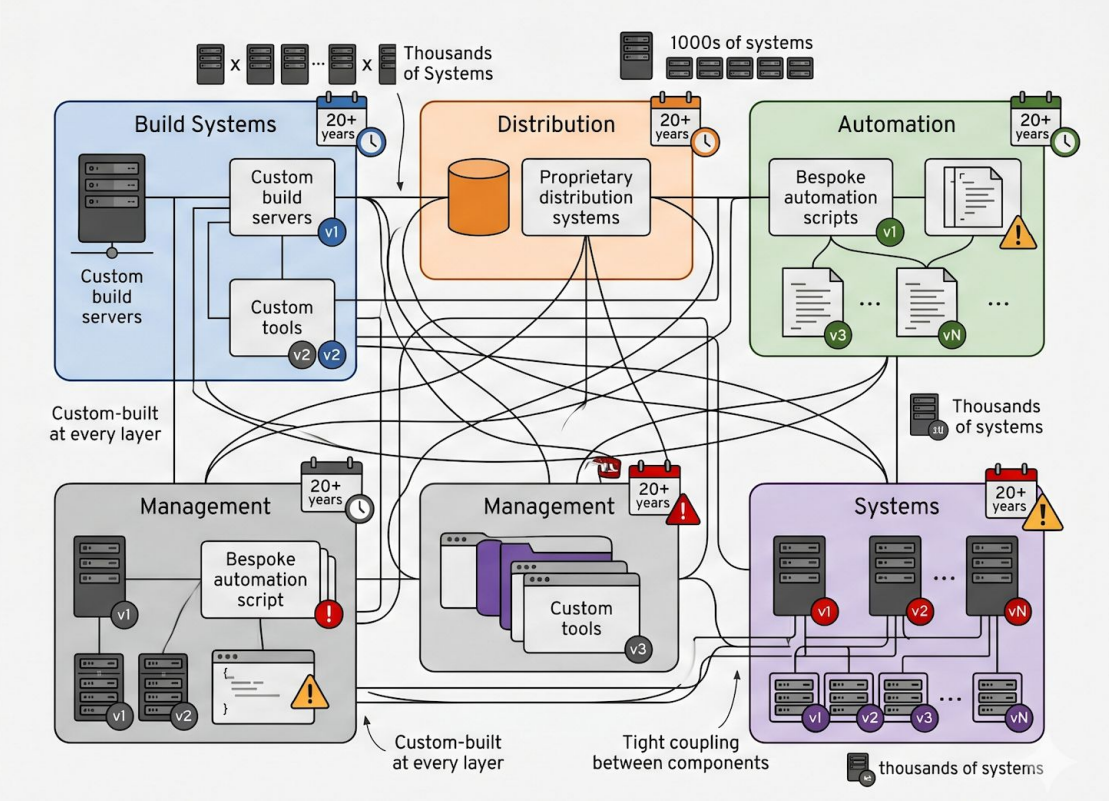
Beyond technical improvements - real operational value

- ➔ **Faster patching cycles = better user experience**
- ➔ **Infrastructure savings from legacy systems**
- ➔ **Improved security posture with immutable infrastructure**

Managing at Enterprise Scale

- Thousands of RHEL systems across global markets.
- 20+ years of customization and technical debt.
- Proprietary distribution mechanisms requiring specialized personnel.
- Isolated toolchains creating organizational silos.

> "We had decades of 'stuff' to account for and established user expectations."



We Chose Image Mode

The Forcing Function → RHEL 9 Migration

Key internal tooling component not supported in RHEL 9

Critical decision: Rebuild custom infrastructure OR fundamentally rethink our approach

We asked: "Why are we maintaining all this custom infrastructure?"

Alternatives We Explored:

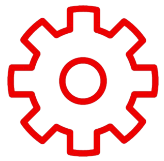
OSTree via Image Builder: Too rigid, long build times, PXE boot challenges

Custom package-based solutions: Would perpetuate technical debt

Silverblue: Didn't match our deployment model

Transformation

From custom tools to container-native workflows



Feb - Apr 2024

Technical evaluation & Partnership

May - Jul 2024

POC - Initial images and testing

Aug - Dec 2024

Prop prep and infra build out

2025 - ongoing

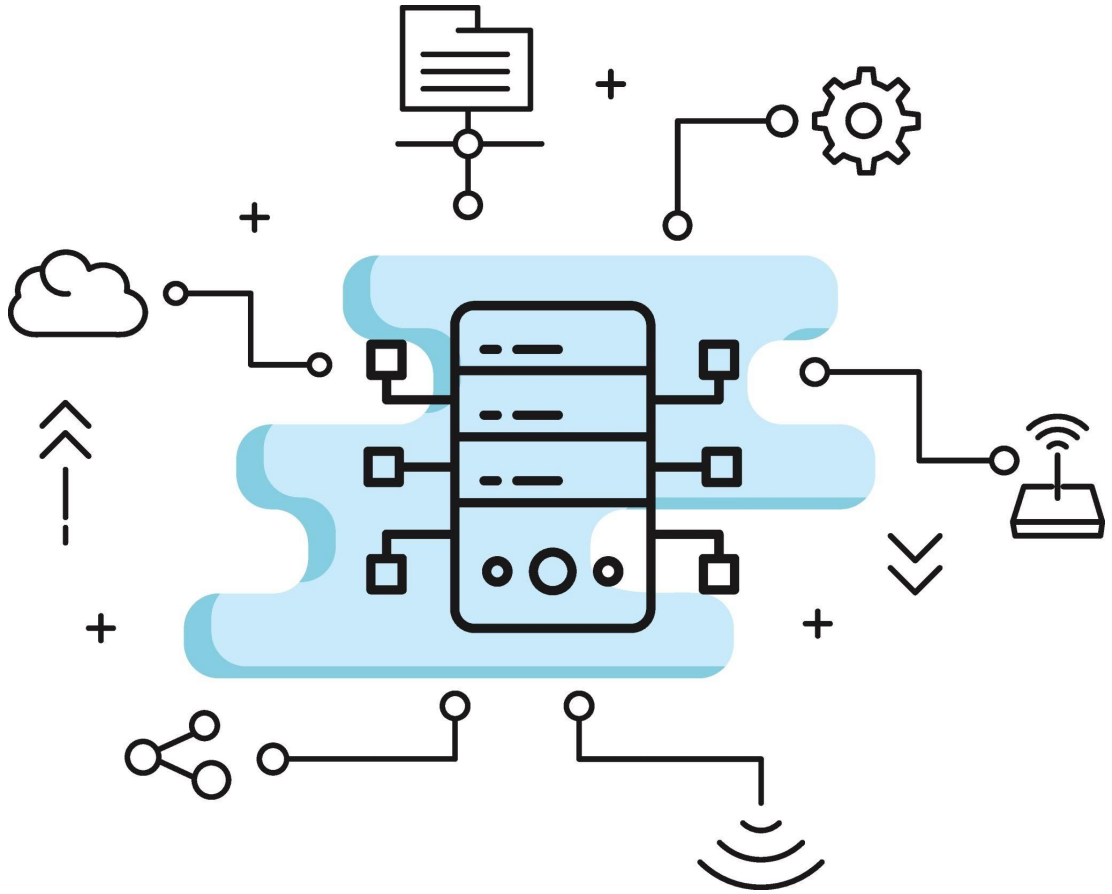
Phased rollout

Legacy Architecture

Custom.

Fragile.

Unsustainable.



Standardization Benefits

Consolidated Infra

Single registry for both OS and App images simplifies disaster recovery.

Modern Automation

Standard Jenkins/GitLab pipelines now handle the operating system.

Skill Transfer

New hires with container experience are productive on day one.

Audit Transparency

Container files provide a versioned, clear view of the OS configuration.

Image Mode Workflow

From Build to Deployment

- **Define:** Start with rhel-bootc base image.
- **Build:** Add system agents and kernel modules.
- **Validate:** Test as a container for fast QA cycles.
- **Publish:** Push to internal registry with environment tags.
- **Scale:** Deploy to metal, cloud and manager via bootc

Package Mode vs Image Mode

	Package Mode	Image Mode
Method	RPM transaction per system	**Atomic image replacement**
Process	Mutates the running system	Stages in background, applies at reboot
Risk	If interrupted, system may be unstable	**Safe - previous image always available**
Rollback	Complex, often unreliable	**Instant - reboot to previous image**
Time	Variable, depends on package count	~90 seconds, **predictable**
Recovery	Usually reinstall the box	Simple, pick boot option

bootc in Action

```
# Check current status
$ bootc status
Image: internal-registry/rhel-soe:prod-v2.3
Boot: A (active)

# Upgrade to new version
$ bootc upgrade
Pulling internal-registry/rhel-soe:prod-v2.4...
Staging new deployment...

# Rollback if needed
$ bootc rollback
Switching to previous deployment...

# Switch to different image entirely
$ bootc switch internal-registry/rhel-soe:test-v3.0
```


Solved: Working with Immutable Infrastructure

The Reality:

- /usr, /opt, and most system paths are read-only at runtime
- Applications need dynamic content, job orchestrators need runtime config

The Solution:

- **Use /var (writable and persistent)**
- User overlays: Enable read/write access for testing
- Custom tooling: Dynamic content management in /var
- Integration points: Runtime configuration for job orchestrators

```
# Before (Package Mode):
/opt/myapp/data
/opt/myapp/config

# After (Image Mode):
/var/opt/myapp/data
/var/opt/myapp/config
# /var is writable and persistent
```

Image Mode benefits + runtime flexibility

Solved: Air-Gapped Image Pipelines

The Problem:

- Security requirements: some build environments are air-gapped
- Cannot fetch container images from external registries
- Need to build bootc images in disconnected environment
- Image builder requires specific setup

The Solution:

- Built custom bootc container configuration
- Internal mirror of required base images
- Disconnected registry infrastructure
- Reliable image pulling mechanisms

Solved: Consistent User IDs

The Problem:

- NFS and shared storage require globally consistent UIDs across a massive fleet. Dynamic assignment doesn't work with immutable images.
- Enterprise requirements: specific UID/GID mappings

The Solution:

- Define user/group mappings at build time in containerfile
- Centralized UID/GID registry integration
- Validate consistency during image creation.

```
FROM rhel10/rhel-bootc:latest

# Define users with consistent UIDs/GIDs from central registry

# Application user (with home directory and shell)
RUN groupadd -g 5001 appgroup && \
    useradd -u 5001 -g appgroup -m -s /bin/bash appuser

# Database user
RUN groupadd -g 5002 dbgroup && \
    useradd -u 5002 -g dbgroup -m -s /bin/bash dbuser
    useradd -u 5003 -g dbgroup  -m -s /bin/bash dbuser

# Service account (no home directory, no login shell)
RUN groupadd -g 5003 svcgroup && \
    useradd -u 5003 -g svcgroup -M -s /bin/nologin svcaccount

# Verify consistent UID/GID assignments
RUN id appuser && id dbuser && id svcaccount
```


Build Flexibility: Custom Kernel Modules

Hardware Support

Integrate specialized drivers not found in upstream RHEL.

Security Modules

Bake proprietary security modules directly into the OS image.

```
FROM rhel10/rhel-bootc:latest

# Install build dependencies
RUN dnf install -y kernel-devel gcc make

# Copy and build custom module
COPY custom-module/ /usr/src/custom-module/
RUN cd /usr/src/custom-module && make && make install

# Load module at boot
RUN echo "custom_module" >> /etc/modules-load.d/custom.conf
```

Full build-time control means modules are tested as part of the total image stack before any deployment.

Efficiency: Multiple Image Variants

Minimal Base

Core system configuration for edge-like internal services.

Standard SOE

The enterprise default for general purpose application hosting.

High-Perf Variant

Specialized libraries and kernel tuning for low-latency trading.

```
# File: Containerfile.specialized
FROM internal-registry/base-soe:latest
RUN dnf install -y performance-tools monitoring-agents
COPY specialized-configs/ /etc/

# File: Containerfile.minimal
FROM internal-registry/base-soe:latest
RUN dnf install -y minimal-tools

# File: Containerfile.highperf
FROM internal-registry/base-soe:latest
RUN dnf install -y tuned-profiles-realtime low-latency-libs
```

Container layering allows us to maintain these with one common base; changes propagate automatically.

Ongoing: Application Vendor Compatibility

The Situation:

- Some third-party applications assume mutable OS
- Installation scripts may attempt writes to read-only paths
- Agents and monitoring tools need adaptation
- Certification and support processes with vendors

Examples of Areas Requiring Work:

- Security agents (endpoint protection)
- Monitoring and observability tools
- Backup solutions
- Compliance scanning tools

Red Hat is actively collaborating with identified partners and ISVs to ensure compatibility with Image Mode deployments

What Worked Well

Red Hat Partnership

Open feedback loop accelerated feature development for our needs.

Early Validation

Metrics-driven POC built organizational buy-in from the start.

Phased Rollout

Don't force everything at once. Learn and adapt between phases.

Flexibility

Pragmatic workarounds for "uncontainerizable" apps were key.

What We'd Adjust

What We'd Do Differently

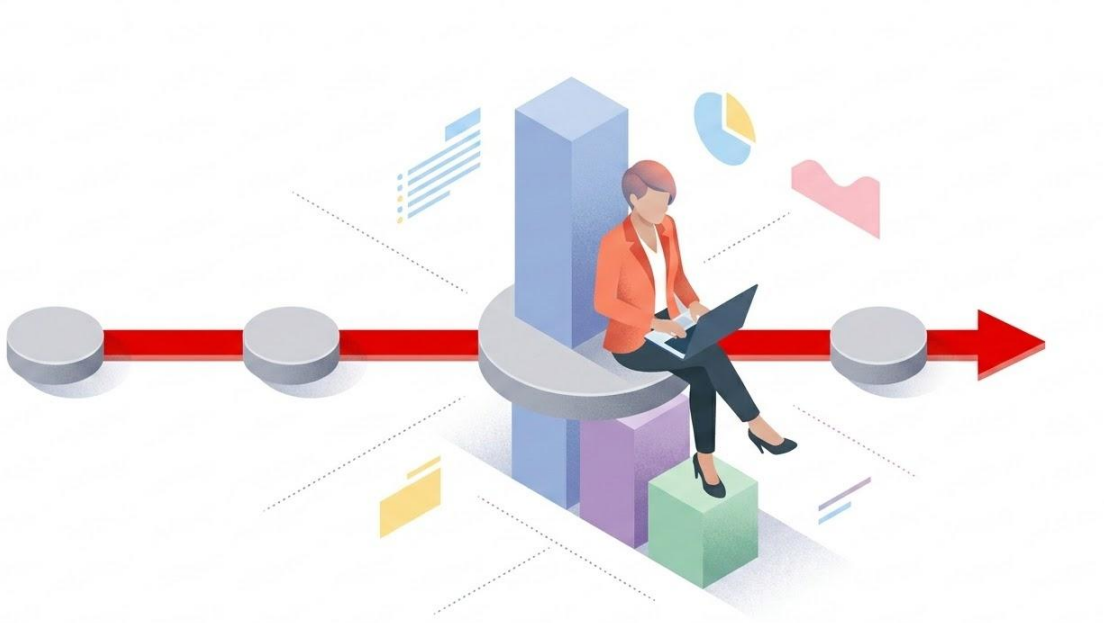
- Involve ISV vendors much earlier.
- Heavier upfront user training.
- Earlier automation of testing frameworks.

What We Did Right

- Strong partnership with Red Hat.
- Performance validation first.
- Full ownership of the Containerfiles.

Every journey has learnings - sharing ours can help your path

What's Next for Our Image Mode Journey



Expanding Adoption

Cont'd Rollout and more teams onboarding

Partner Compatibility

Completing ISV certifications

Process Automation

Enhanced build pipelines and testing

Team Enablement

Building internal expertise

Is Image Mode Right for You?

Ideal Profile

Scale environments, GitOps focus, container native workflows.

Considerations

Investment in redesign, training and paradigm shifts.

Start small, validate benefits, scale with confidence.

Why This Matters for the Industry

- ➔ **Contain drift and accelerate delivery**
- ➔ **Proven Image Mode viability at enterprise scale**
- ➔ **Accelerate ISV and partner adoption**

Image Mode is ready for complex enterprise environments - with the right partnership!

Looking forward

Image mode for RHEL - Roadmap

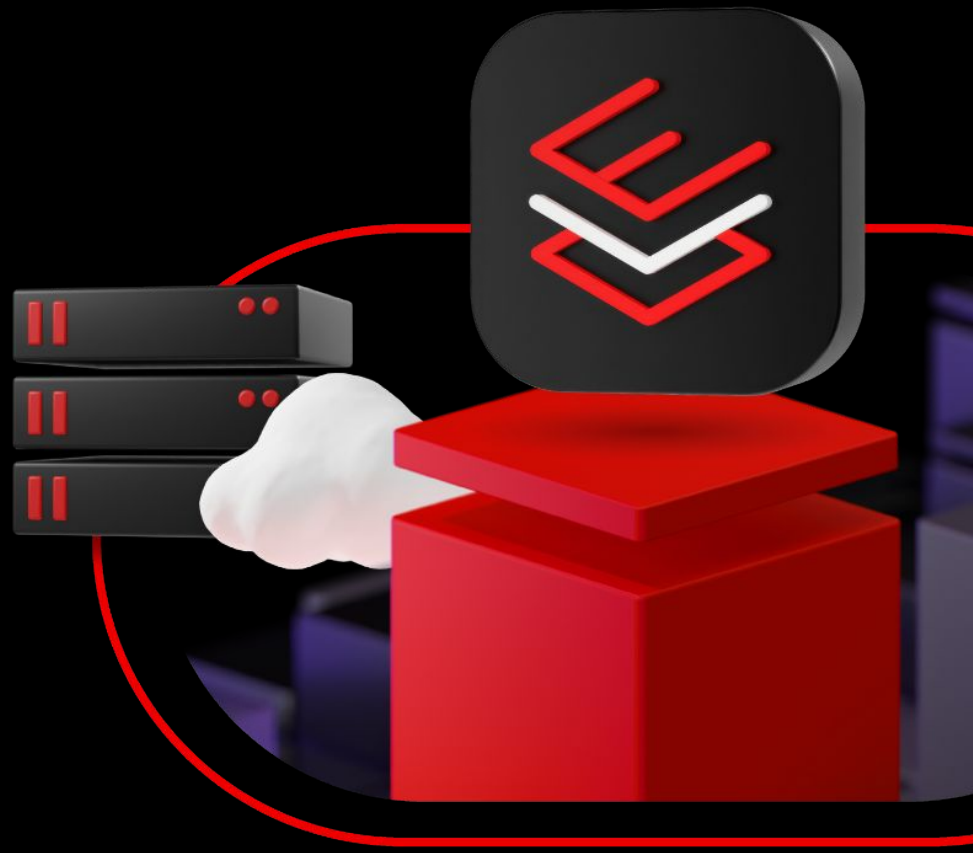
- **Integrity Image Sealing** enables chain of trust from firmware to runtime.
- **Download-only upgrades** pre-fetch updates, plan maintenance schedule
- **Package mode to image Mode** migration tools
- Bootc **Cloud Images**, minimal images
- More...

Give image mode for Red Hat Enterprise Linux a try

Visit red.ht/imagemode

Get hands on with image mode at red.ht/im-rhel

Red Hat booth for Image Mode demos



The Red Hat Image Mode team wants to hear from *you!*

Help shape what comes next.

Scan the QR code or go to:
red.ht/userfeedback

Share your pain points, experiences, and ideas.
Visit the UX team at the RHEL booth

This session qualifies for a shirt!

Pick up a redemption coupon at the back of the session as you exit.

Redeem at the RHEL Booth in the EXPO HALL

* WHILE SUPPLIES LAST *

Red Hat

Thank you

linkedin.com/company/red-hat

facebook.com/redhat

instagram.com/red_hat

x.com/RedHatSummit

Red Hat